PROJECT REPORT

Table-Tennis Stroke Recognition from Wrist-Worn Smartphone Sensors

Rebuilding a gesture-recognition pipeline so that what works offline also works in real time

Marco Gallinaro
October 2026

Tools: MATLAB R2026a (Signal Processing, Statistics and Machine Learning, Deep Learning Toolboxes, Simulink, Simulink Coder, Embedded Coder, Simulink Support Package for Android Devices)

1. Summary

This project started from a university group project (course of Automation and Safety, UCBM, 2024) whose goal was to recognize table-tennis strokes from the accelerometer and gyroscope of a smartphone worn on the wrist. The original system reached **98% accuracy offline** but only **40% in the real-time test** on the phone.

In this follow-up I rebuilt the whole pipeline on my own, with one guiding rule: **every sample that reaches the network must go through exactly the same processing in training, in testing and in real time**. Along the way I found and fixed several problems that explained the original gap: corrupted labels in the dataset, a wrong assumption about the sensor sampling rate, filters that could not run in real time, windows cut differently offline and online, and a validation scheme that leaked information between training and test. In a live test on the phone, the new system recognized 55 of 60 strokes (91.7%).

Key result	Value
Test accuracy on subjects never seen in training	**95.5%** (969 / 1015 windows)
Live test on the phone	**91.7% (55 / 60 strokes): serve 90%, forehand 90%, backhand 95% (one session, author)**
Classes	Serve (S), forehand topspin (TPD), backhand topspin (TPR)
Model	1D convolutional network, 11,683 parameters
Online detector	Sample-by-sample, causal, latency ≈ 0.5 s
Online vs offline	Streaming simulation reproduces the offline windows and classes exactly
Deployment status	Running stand-alone on an Android phone (Simulink, generated code)

2. Background and goal

2.1 The original project

A group of four students recorded volunteers performing three table-tennis strokes and a "hand at rest" class, with a smartphone strapped to the wrist. The group compared two approaches: hand-crafted features with K-NN and Random Forest, and neural networks (a feed-forward network on the flattened window and an LSTM) applied directly to the signal. My part was the neural networks and the real-time Simulink model.

Model	Offline accuracy	Real-time accuracy
Feed-forward network (FFNN)	98.2%	40% (6 / 15 trials)
Random Forest	98.0%	62.5% (10 / 16 trials)

Table 1. Results of the original group project.

2.2 Goal of the rework

- Understand **why** the offline result did not transfer to real use.
- Rebuild the pipeline from scratch, writing every line of code myself, with each step verified before moving to the next.
- Obtain an **honest** accuracy figure, measured on people the network has never seen.
- Produce a real-time detector whose behaviour is identical to the offline pipeline.

3. Data

- **11 subjects**, 4 activities (serve, forehand topspin, backhand topspin, rest), 5 recordings of 60 s each per subject and activity: 220 recordings in the file.
- 6 channels: 3-axis accelerometer and 3-axis gyroscope.
- Each recording is stored as 24,001 rows per sensor, i.e. 60 s at 400 rows per second (see Section 4.2 for what the sensor actually delivered).
- Subjects are identified here by anonymous codes P01–P11. The raw data are not public.

4. Challenges and how they were solved

Each challenge below is described as: what was observed, what was done, and the outcome.

4.1 Corrupted labels in the dataset

Observation. Plotting the four activities of one subject, the "rest" class showed regular strokes with magnitude up to 40 and a gyroscope trace identical to the backhand.

Check. An exact comparison (`isequal`) of every recording with every other one, about 24,000 pairs.

Comparison	Identical accelerometer	Identical gyroscope
Rest (F) vs serve (S)	0 / 55	0 / 55
Rest (F) vs forehand (TPD)	0 / 55	0 / 55
Rest (F) vs backhand (TPR)	**55 / 55**	**55 / 55**

Table 2. The "rest" class is an exact copy of the backhand class. One subject also has File5 identical to File4 in TPD and TPR.

Outcome. The rest class and the duplicated files were excluded. This also explains a symptom of the original project: the network separated "backhand" from "rest" in 92% of cases although they were the same data. It was learning **how the windows were cut** (around peaks for strokes, at fixed intervals for rest), not the gesture.

Rest class in the new design. Instead of a fourth class, "no stroke" is decided by the peak detector: a hand at rest never crosses the threshold and never reaches the network.

4.2 The real sampling rate was about 50 Hz, not 400 Hz

Observation. The filtered signal inside the first extracted window looked like a staircase: constant for about 8 rows, then a jump.

Check on the whole dataset (330 recording–sensor pairs):

Check	Result	Interpretation
Length of the constant steps	8 rows: 43%; 7: 22%; 9: 5.5%; 1–2: 20%	A new value about every 20 ms, with small jitter
Position of the changes modulo 8	Uniform	Updates are asynchronous with the 400 Hz grid
x, y, z changing together	≥ 98%	One sensor event updates all three axes
Updates per second, every recording	51.2–51.7	Homogeneous dataset, about 51.5 values per second

Table 3. The file stores each sensor value until the next one arrives (zero-order hold).

Consequences. The 800-sample windows of the original project ("2 s at 400 Hz") contained only about **100 real values**; the network received 8 times more numbers with no extra information. On the phone, the sensors deliver about 50 values per second anyway.

Solution. All data are decimated to 50 Hz (one row in 8) **right after loading, before filtering**, so filter, peak detection and windows work at the true rate of the sensor, the same rate the phone provides online. All parameters expressed in samples are derived from `fs`.

Lesson. A first attempt that changed only the value of `fs`, without resampling the signal, silently broke every parameter (the filter cut at 4 Hz instead of 0.5 Hz, the minimum distance between strokes became 8 s). The value of `fs` must describe the signal that is actually there.

4.3 Filtering that can run in real time

- High-pass Butterworth filter, 2nd order, 0.5 Hz: removes gravity (phone orientation) and drift, keeps the movement.
- **Causal `filter`, not `filtfilt`.** `filtfilt` runs forwards and backwards and needs the whole signal: it cannot be reproduced online. A causal filter can be applied sample by sample, carrying its internal state, and gives exactly the same output as on the whole recording.
- The price is a start-up transient of about 2 s, which is discarded (first 5 s offline, first 3 s online).

4.4 Finding the strokes

Magnitude instead of a single axis

Peaks on a single acceleration axis left too small a margin between weak strokes (about 3) and noise (up to 2.5). The **magnitude of the acceleration** does not depend on how the phone is oriented and collects the movement of all three axes: strokes stand at 11–22, side bumps mostly below 9. The magnitude is used only to **find** strokes; the windows are cut from the six original channels, because the direction of the movement is what separates forehand and backhand.

A threshold that adapts to the player

Amplitude varies enormously: the standard deviation of the magnitude goes from 1.8 to 9.7 across serve recordings, and **the same person varies between files as much as different people do**. A fixed threshold of 10 finds all 29 strokes in the strongest recording and none in the weakest, whose strokes are regular but reach only 6–10.

I looked for a statistic proportional to the "right" threshold (read from the plots) in three extreme recordings:

Recording	Right threshold	÷ std	÷ median	÷ 90th perc.	÷ 95th perc.
P03_File1 (weakest)	5–6	2.8–3.4	**2.3–2.7**	0.9–1.1	0.8–1.0
P01_File1	9–11	2.6–3.1	**2.2–2.7**	1.0–1.2	0.7–0.9
P09_File2 (strongest)	8–19	0.8–2.0	**1.3–3.1**	0.3–0.8	0.3–0.6

Table 4. Only with the median do the three ranges overlap: the median measures the background between strokes, while the standard deviation and high percentiles are dominated by the strokes themselves.

Chosen rule: **threshold = 2.0 × median of the magnitude**, minimum distance 1 s between strokes. It finds every stroke counted by eye in the three reference recordings (24, 28 and 29) with no false peaks and no per-recording tuning.

Automatic quality control

Every recording is checked through the rhythm of the strokes: an interval longer than 1.6 × the median interval suggests a missed stroke; one shorter than 0.6 × suggests a false peak. Serve: 50 of 55 recordings without alarms. Ranking the recordings by severity showed that problems cluster **by person** (technique, strength, phone position), not at random.

Where the rule fails, and what was decided

- **Continuous movement.** In two backhand recordings of one subject the arm never returned to rest: the median fell in the middle of the oscillation and the threshold rose above almost every stroke (4–6 found out of about 45). These two recordings were excluded (less than 4% of the backhand data) and the limitation is declared: a player who moves like this would also defeat the online detector.
- **Decreasing amplitude.** In some recordings the strokes weaken during the minute and the weakest are missed. These recordings were **kept**: the data are fine, it is the method that is imperfect, and removing them would make the dataset easier than reality.
- **Data-quality policy.** For building the training set the two errors do not weigh the same: a missed stroke costs one window, a false peak creates a window without a stroke labelled as a stroke, which teaches the network something false. So: **missed stroke → accepted; false peak → removed**.
- **False-peak removal.** If two consecutive peaks are closer than 0.6 × the median interval, the lower one is removed, one at a time, recomputing the intervals each time. The removed peaks are about half the height of real strokes, consistent with the return movement of the arm. At 50 Hz: 2 peaks removed in serve, 7 in forehand, 6 in backhand.

4.5 Windows centred on the stroke

- One window per stroke: **1 s centred on the peak** (25 samples before, the peak, 24 after) → 50 × 6 samples. 1 s and not 2 s because the closest strokes are 1.2–1.3 s apart.
- Windows that would overlap the start-up or the end of the recording are discarded instead of being truncated: **4710 windows** saved, 55 discarded (about 1%). Classes are balanced: S 1658, TPD 1536, TPR 1516.
- Alignment check: in **4703 of 4710 windows (99.85%)** the maximum is exactly at sample 26. The 7 exceptions come from how `MinPeakDistance` suppresses peaks (from the highest down, so a peak already removed can no longer remove its neighbour). They were kept: the online detector behaves the same way.

4.6 Data leakage and an honest validation

The original project split windows **at random** into training and test. Each person strikes in a personal way, so about 40 forehands of a subject ended up in training and 10 in test: the network recognized them because they resembled strokes **of the same person** already seen. This is data leakage, and "very high offline, collapse online" is its typical symptom. In real use the system is worn by someone the network has never seen.

Set	Subjects	Windows
Training	P01, P03, P05, P06, P08, P09, P11	2835 (60%)
Validation (early stopping)	P02, P10	860 (18%)
Test	P04, P07	1015 (22%)

Table 5. Subject-wise split. P04 is the author, who also performed the live test: the model used online is never trained on his data. An assertion in the code stops the script if any window belongs to zero or two sets.

- **Normalization:** z-score per channel (the six channels have different units and scales), with mean and standard deviation computed **on the training set only** and then frozen. They are part of the model: validation, test and online data are normalized with the same 6 + 6 numbers. Normalizing each window on its own was rejected because it would erase the stroke amplitude.
- **Distribution shift:** on the training scale, subject P04 has accelerations on x and y almost twice the strongest training subject. Decision: train as is, evaluate each test subject separately, and use amplitude augmentation only if needed.

4.7 A model that does not depend on where the stroke is

The original FFNN received the window flattened into 4800 values: each input is tied to a precise instant, so a stroke shifted in time is, for the network, an input never seen before. I chose a **1D convolutional network**: its filters slide along time and recognize a shape wherever it is, and global average pooling summarizes "how much of each pattern is present" regardless of position. It also has few parameters, which matters with 2835 training windows.

Layer	Purpose
Sequence input, 6 channels × 50 samples	Accelerometer + gyroscope over time
Conv 1D, 32 filters of 5 samples (0.1 s) + batch norm + ReLU	Simple local shapes
Max pooling 2	50 → 25 samples, more tolerance to shifts
Conv 1D, 64 filters of 5 samples + batch norm + ReLU	More complex patterns
Global average pooling	Removes the time axis: position-independent
Dropout 0.3 → fully connected (3) → softmax	Regularization and class probabilities

Table 6. Architecture: 11,683 parameters in total, fewer than half of the first layer alone of the original FFNN (about 30,000).

Training: Adam, mini-batch 64, shuffling every epoch, validation every epoch with early stopping (patience 5) and the best-validation network returned. Training stopped at epoch 18; the best network (epoch 13) reached about 99% on the validation subjects. Training took 36 s on a GPU.

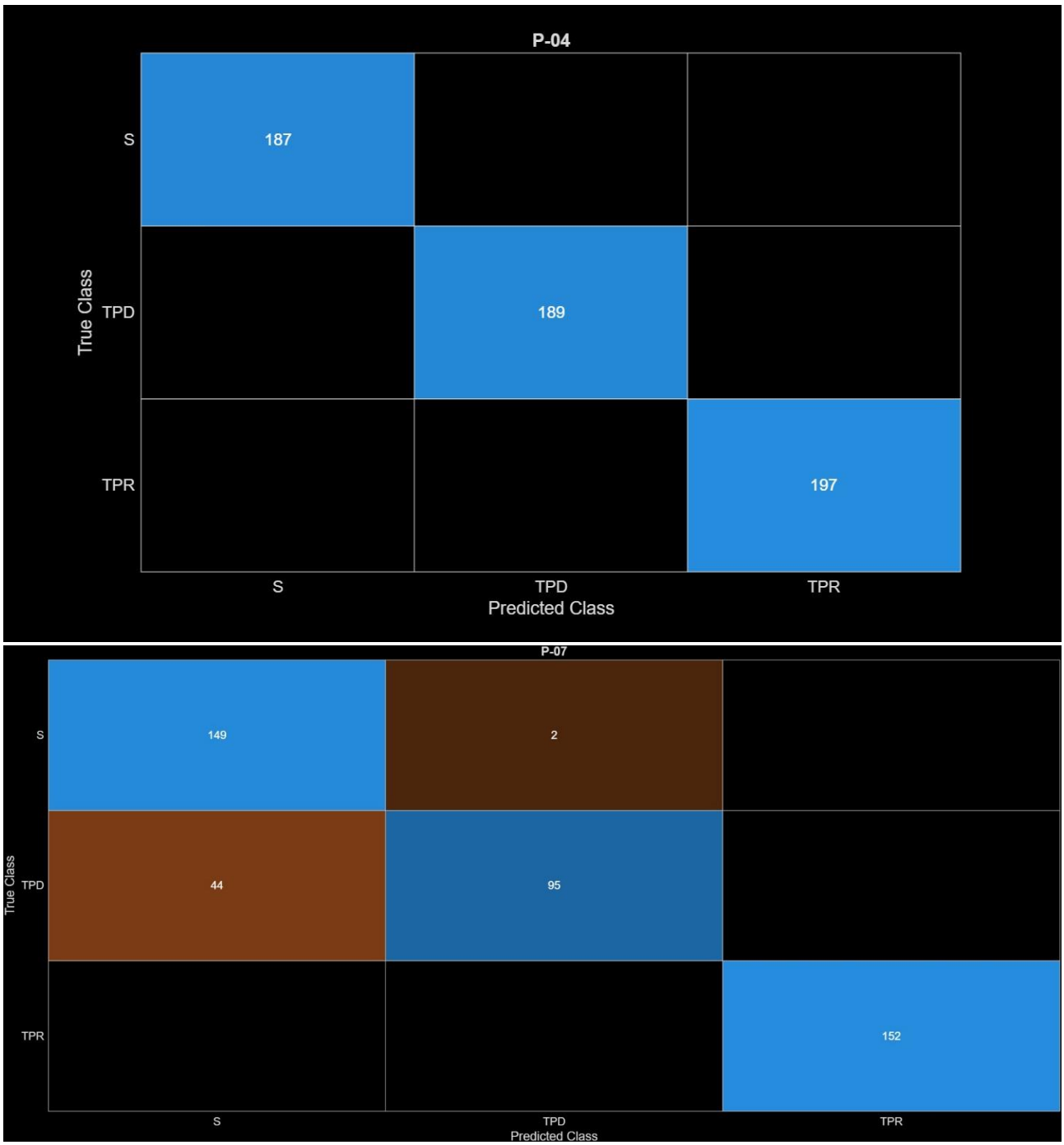


Figure 1 Confusion Matrix for test subjects (P04 and P07)

4.8 A real-time detector identical to the offline pipeline

Online there is no whole recording: samples arrive one at a time, every 20 ms. The detector was designed so that its windows are the same as the training windows.

- **Causal filter with memory:** each new sample is filtered as soon as it arrives, continuing from the previous filter state.
- **Sliding buffer** of the last 50 filtered samples. At every sample the detector asks whether the **central sample (row 26)** is a stroke: the maximum of the buffer, above the threshold, and at least 1 s after the previous stroke. If so, **the buffer itself is the window**, already centred on the peak, exactly as in training. The price is a latency of 24 samples (about 0.5 s).

- **Hybrid threshold:** offline the threshold uses the median of the whole recording, which does not exist online. Online it is `max(2 × median of the last 10 s, 2)`. The adaptive part follows the player; the floor of 2 prevents the threshold from collapsing during long pauses.

Activity	Lowest offline threshold	Median offline threshold	Median "rest" level (3–4 s)
Serve	3.99	7.16	0.44
Forehand	7.38	15.24	0.66
Backhand	10.80	15.92	1.70

Table 7. Calibration of the threshold floor on training subjects only. The floor (2) sits about four times above rest and at half the lightest serve. Many recordings were already in motion at 3–4 s, so this "rest" level is only indicative.

- The network runs **only when a stroke is detected**, not at every sample.

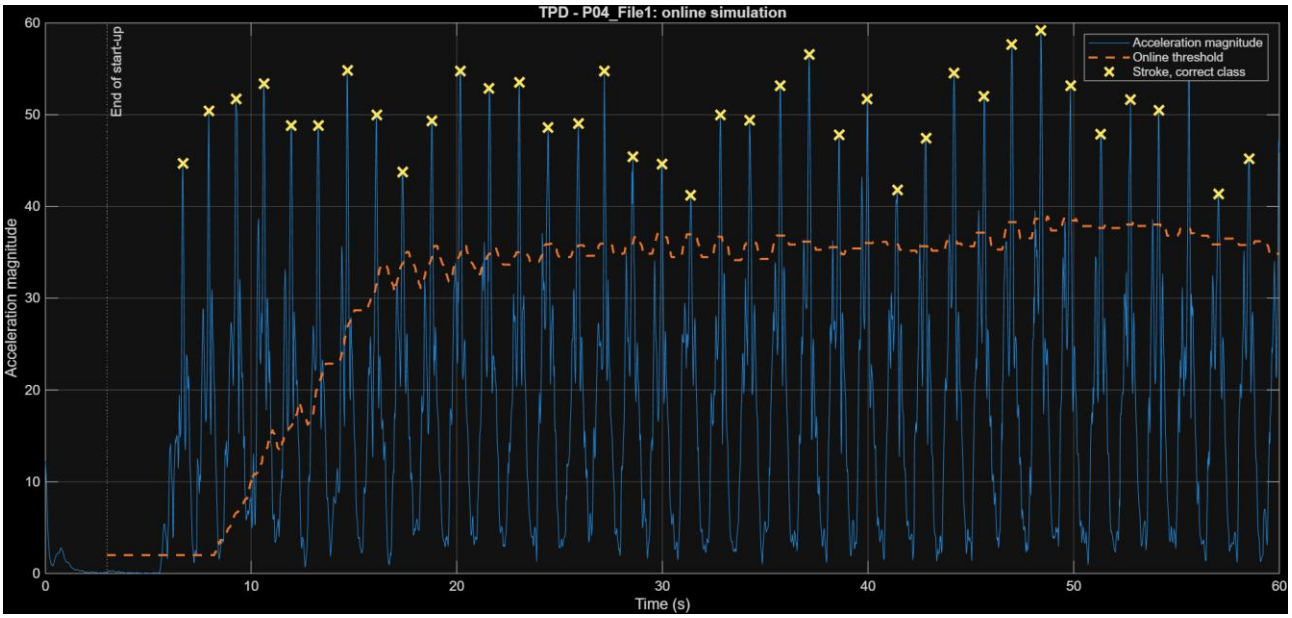
4.9 From a script to a function that Simulink can call

Simulink does not run a loop over a recording: it calls a function at every new sample. The detector was therefore first written as a plain loop and verified, then moved piece by piece into a function (`passoOnline`) that keeps its memory (filter state, buffer, magnitude history, last stroke) in persistent variables. After each piece, the function was compared with the loop on a full test recording:

Piece moved into the function	Check	Result
Causal filter with state	Max difference vs filtering the whole signal	0
Magnitude history and window buffer	Max difference on all 2952 buffers	0
Start-up and hybrid threshold	Max difference at every sample	0
Peak test	Same stroke instants as the loop	identical
Normalized window and network (outside)	Same predicted classes as the loop	identical

Table 8. Step-by-step verification of the real-time function.

The final function has only two outputs, "stroke detected" and the normalized window; the network stays outside (in Simulink, a Predict block), and the function uses only operations suitable for code generation.



4.10 From the function to an app on the phone

The verified function was placed inside a Simulink model, which was first tested on the PC and then compiled into an app running on an Android phone.

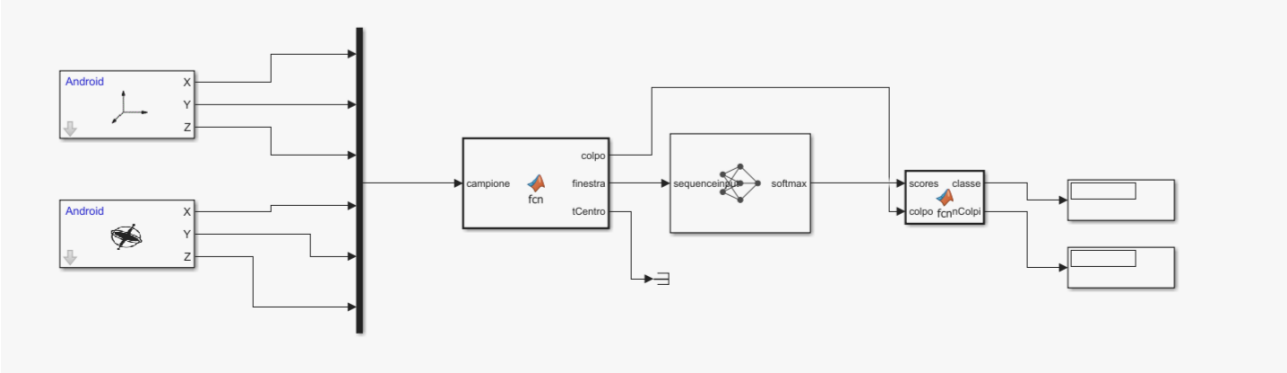


Figure 2 Screenshot of the Simulink model

Figure 1. Simulink model running on the phone. The sensor blocks read the accelerometer and gyroscope every 20 ms; the MATLAB Function block runs the stroke detector (passoOnline); the Predict block runs the trained CNN; the last block keeps the class of the latest stroke and counts the strokes; both are shown on the phone screen.

How the model works. Every 20 ms the phone delivers one new sample of the six channels. The model:

- combines them in the same order used in training (accelerometer x, y, z, then gyroscope x, y, z);
- passes the sample to the detector, which filters it, adds it to the 50-sample buffer and checks whether a stroke has just occurred;
- when a stroke is found, sends the normalized window to the network, which returns the probability of each class;
- shows the most likely class (1 = serve, 2 = forehand, 3 = backhand) and keeps it on screen until the next stroke, together with a stroke counter.

The filter coefficients and the normalization statistics are loaded automatically when the model starts, from the same file saved after training. The phone therefore uses exactly the same numbers as the training pipeline.

Verified on the PC before the phone. The sensor blocks were first replaced by a block that replays a recorded session at 50 Hz, one sample every 20 ms. The model found the same 28 strokes, at the same instants, with the same classes as the MATLAB code. Only then were the sensor blocks put back; the rest of the model was not changed.

Same units as the training data. In the recordings the gyroscope reaches about 13 rad/s and the accelerometer about 64 m/s² during strokes. These are the units of the Android sensor blocks, so the network receives values on the scale it was trained on.

Problems found while moving to the phone

Problem	Cause	Solution
Build stopped with “invalid folder name”	The Android build tools do not accept folder paths with spaces	Working folder moved to a path without spaces
The app was built but not installed (“no matching ABIs”)	The app had been compiled for 32-bit processors; the phone accepts only 64-bit apps	Target set to Android (64-bit)

Problem	Cause	Solution
The phone blocked the installation	Security check on apps installed from a computer and built for an older Android version	Installation confirmed manually on the phone
Code generation stopped on the detector	The phone sensors deliver single-precision values, while the detector stores double-precision values; generated code cannot change type	Each sample converted to double as it enters the detector, as in the PC version
The network received the window with rows and columns swapped	The Predict block expects channels on the rows unless told otherwise	Input format set to time × channels, as in MATLAB
The class was not visible on screen	It lasted a single 20 ms step	The last class is held until the next stroke, and a counter shows each new detection
Live monitoring from the PC failed ("device not connected to a Wi-Fi network")	Following the model from the PC requires a Wi-Fi link between phone and PC, which did not work on every network	The app was run stand-alone, with the class and the stroke counter shown on the phone screen

Running on the phone. The model is compiled into an app and installed on the phone through the USB cable. Two modes are possible: one in which the PC follows the app's values in real time over Wi-Fi, and one in which the app runs on its own. The live test used the stand-alone mode: the class of the last stroke (1 = serve, 2 = forehand, 3 = backhand) and the stroke counter are shown directly on the phone screen. Once the app has started, the cable can be removed and the phone worn on the wrist, with no computer nearby.

Comparison with the original online model. An early version of the group's Simulink model (December 2024) read the sensors every 0.1 s (10 Hz). Its filters were designed for 400 Hz and applied to 60-second blocks, each block restarting the filters, and it produced one prediction about every minute. The new model processes every sample as it arrives and gives a class about 0.5 s after each stroke.

5. Results

5.1 Offline, on subjects never seen in training

True class	Correctly recognized	Errors
Serve (S)	99.4%	2 → forehand
Forehand topspin (TPD)	86.6%	44 → serve
Backhand topspin (TPR)	100%	0
Overall	**95.5%** (969 / 1015)	46

Table 9. Test-set results (subjects P04 and P07).

- The backhand is never confused: it is played with the back of the hand and the signals have opposite signs. Serve and forehand are the most similar gestures physically, and all errors are between them.

- **Per subject:** P04 100% (573 / 573), P07 89.6% (396 / 442). 39 of P07's 44 errors fall in just two forehand recordings (20 / 27 and 19 / 31 wrong), while the other three forehand files have 5 errors out of 81. These files are not duplicates of serve recordings; the most likely explanation is a "half-way" forehand in the first sessions. Excluding them, accuracy on the other test recordings is about 99.3%. They were kept.
- The subject expected to be at risk (P04, with amplitudes outside the training range) was classified perfectly: global amplitude statistics did not predict where the network fails.

The 95.5% is lower than the 98% of the original project, and that is the point: this figure is measured on people the network has never seen, while the original one was inflated by data leakage.

5.2 Online (streaming simulation)

- Test forehand recording of P04: every stroke found offline was found online at the same instant, except the last one (it occurs less than 0.5 s before the recording ends, so it cannot be confirmed in time). No false peaks; secondary bumps between strokes stay below the adaptive threshold. All strokes classified as forehand.
- Test forehand recording of P07 (one of the two difficult files): 31 strokes, 12 correct, **exactly as offline**. The online chain reproduces the offline result; the low score belongs to the data discussed above, not to the detector.
- The real-time function gives the same stroke instants and classes as the reference loop (Table 8).

5.3 Live test on the phone

The app was tested in real time with the phone strapped to the wrist, by subject P04 (the author), who was never part of the training data. Twenty strokes per class were performed; after each stroke, the class shown on the phone was recorded. A stroke counts as correct only if it was both detected and assigned to the right class.

Stroke performed	Strokes	Correct	Accuracy
Serve (S)	20	18	90%
Forehand topspin (TPD)	20	18	90%
Backhand topspin (TPR)	20	19	95%
Total	**60**	**55**	**91.7%**

Table 10. Live test on the phone (one session, one subject).

- Real-time accuracy (91.7%) stays close to the offline result on unseen subjects (95.5%), instead of collapsing as in the original project (40%). This is the main goal of the rework: what works offline now also works in real time.
- The backhand is the best-recognized stroke, as offline.
- With 20 strokes per class, each percentage has a margin of a few points: the result shows a clear difference from the original 40%, not a precise accuracy figure.
- These figures come from a single session by one person, and offline P04 was already classified perfectly. They show that the system works in real time, but they are not yet a general accuracy figure (see Section 6).

5.4 Original project vs this rework

Aspect	Original project	This rework
Classes	4, of which one was a copy of another	3 strokes; "no stroke" decided by the detector
Sampling rate	Assumed 400 Hz	True rate ≈ 50 Hz, used everywhere

Aspect	Original project	This rework
Filtering	Whole recording offline, each 2 s block alone online	Causal filter, identical offline and online
Windows	2 s around the peak offline; fixed 2 s blocks online	1 s centred on the peak, offline and online
Validation	Random split of windows (leakage)	Split by subject, normalization from training only
Model	FFNN on 4800 flattened values	1D CNN, 11,683 parameters
Offline accuracy	98.2% (inflated)	95.5% on unseen subjects
Online	40% (15 live trials)	91.7% (60 live strokes, one session)

Table 11. What changed.

6. Limitations

- **Only two test subjects.** The result depends on which two are chosen. A leave-one-subject-out evaluation (11 trainings, each with a different test subject) is planned to report mean and spread.
- **Three classes.** A real "rest" class was not available; rest is handled by the detector. If the live test shows false alarms, a fourth "no stroke" class built from false peaks can be added.
- **Continuous movements** (some backhands) can defeat a median-based threshold. A prominence-based peak criterion is a candidate improvement.
- **Threshold floor** to be confirmed with a dedicated recording at rest with the phone on the wrist.
- **Limited live test.** The live result comes from one session by one person (the author), with 20 strokes per class. A test on other players, across several sessions, is needed before stating a general real-time accuracy.

7. Lessons learned

- **Look at the data before modelling.** Two of the biggest problems (a copied class, a wrong sampling rate) were found by plotting and checking the raw data, not by tuning the network.
- **A very high offline accuracy is a warning sign** when the online result disagrees. Validation must reproduce the real use: here, a new person.
- **Design for streaming from the start.** Causal filters, the same window logic and the same normalization online and offline remove an entire class of train/deploy mismatches.
- **Verify every step against a reference.** Moving the detector into a function one piece at a time, with an exact comparison after each piece, caught errors that would have been very hard to find in Simulink.
- **Decide data-cleaning rules before seeing the results**, and declare the limits of the method instead of removing the data that expose them.

8. Code and data

The MATLAB code is available on GitHub: anonymization of the dataset, preprocessing pipeline, CNN training, online simulation, the real-time function with its debug version and the Simulink model for the phone. The recordings belong to the volunteers and are not published; subjects are identified only by anonymous codes.

Repository: MGallinaro/Table-Tennis-Stroke-Recognition-from-Wrist-Worn-Smartphone-Sensors: Real-time recognition of three table-tennis strokes (serve, forehand topspin, backhand topspin) from the accelerometer and gyroscope of a smartphone worn on the wrist. The project uses MATLAB and a 1D convolutional neural network, and runs in Simulink on an Android phone.